

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
```

```
df=pd.read_csv(r"C:\Users\304690\Downloads\Customer Churn.csv")
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure
0	7590-VHVEG	Female	0	Yes	No	1
1	5575-GNVDE	Male	0	No	No	34
2	3668-QPYBK	Male	0	No	No	2
3	7795-CF0CW	Male	0	No	No	45
4	9237-HQITU	Female	0	No	No	2

	MultipleLines	InternetService	OnlineSecurity	...
0	No phone service	DSL	No	...
1	No	DSL	Yes	...
2	No	DSL	Yes	...
3	No phone service	DSL	Yes	...
4	No	Fiber optic	No	...

	TechSupport	StreamingTV	StreamingMovies	Contract
0	No	No	No	Month-to-month
1	No	No	No	One year
2	No	No	No	Month-to-month
3	Yes	No	No	One year
4	No	No	No	Month-to-month

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No

1	Mailed check	56.95	1889.5	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes

[5 rows x 21 columns]

#Understanding the data information

df.info()

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 7043 entries, 0 to 7042

Data columns (total 21 columns):

#	Column	Non-Null Count	Dtype
0	customerID	7043 non-null	object
1	gender	7043 non-null	object
2	SeniorCitizen	7043 non-null	int64
3	Partner	7043 non-null	object
4	Dependents	7043 non-null	object
5	tenure	7043 non-null	int64
6	PhoneService	7043 non-null	object
7	MultipleLines	7043 non-null	object
8	InternetService	7043 non-null	object
9	OnlineSecurity	7043 non-null	object
10	OnlineBackup	7043 non-null	object
11	DeviceProtection	7043 non-null	object
12	TechSupport	7043 non-null	object
13	StreamingTV	7043 non-null	object
14	StreamingMovies	7043 non-null	object
15	Contract	7043 non-null	object
16	PaperlessBilling	7043 non-null	object
17	PaymentMethod	7043 non-null	object
18	MonthlyCharges	7043 non-null	float64
19	TotalCharges	7043 non-null	object
20	Churn	7043 non-null	object

dtypes: float64(1), int64(2), object(18)

memory usage: 1.1+ MB

Replacing the blank with zero along with datatype

```
df['TotalCharges']=df['TotalCharges'].replace(' ','0')
```

```
df['TotalCharges']=df['TotalCharges'].astype('float')
```

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                7043 non-null   object
2   SeniorCitizen         7043 non-null   int64
3   Partner               7043 non-null   object
4   Dependents            7043 non-null   object
5   tenure                7043 non-null   int64
6   PhoneService          7043 non-null   object
7   MultipleLines          7043 non-null   object
8   InternetService       7043 non-null   object
9   OnlineSecurity        7043 non-null   object
10  OnlineBackup           7043 non-null   object
11  DeviceProtection      7043 non-null   object
12  TechSupport           7043 non-null   object
13  StreamingTV           7043 non-null   object
14  StreamingMovies       7043 non-null   object
15  Contract              7043 non-null   object
16  PaperlessBilling      7043 non-null   object
17  PaymentMethod         7043 non-null   object
18  MonthlyCharges        7043 non-null   float64
19  TotalCharges          7043 non-null   float64
20  Churn                 7043 non-null   object
dtypes: float64(2), int64(2), object(17)
memory usage: 1.1+ MB

```

Checking null value

```

df.isnull().sum()

customerID      0
gender          0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines   0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0

```

```
Contract      0
PaperlessBilling  0
PaymentMethod  0
MonthlyCharges  0
TotalCharges   0
Churn          0
dtype: int64
```

Checking duplicacy in data

```
df.duplicated().sum()
```

```
0
```

```
df.describe()
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7043.000000	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692	2279.734304
std	0.368612	24.559481	30.090047	2266.794470
min	0.000000	0.000000	18.250000	0.000000
25%	0.000000	9.000000	35.500000	398.550000
50%	0.000000	29.000000	70.350000	1394.550000
75%	0.000000	55.000000	89.850000	3786.600000
max	1.000000	72.000000	118.750000	8684.800000

Converting 0 to No and 1 to Yes

```
df['SeniorCitizen']=df['SeniorCitizen'].replace({0:"No",1:"Yes"})
df['SeniorCitizen']
```

```
0      no
1      no
2      no
3      no
4      no
```

```
...
```

```
7038   no
7039   no
7040   no
7041  Yes
7042   no
```

```
Name: SeniorCitizen, Length: 7043, dtype: object
```

```
df.head(10)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure
PhoneService \						
0	7590-VHVEG	Female	no	Yes	No	1
No						
1	5575-GNVDE	Male	no	No	No	34
Yes						
2	3668-QPYBK	Male	no	No	No	2
Yes						
3	7795-CF0CW	Male	no	No	No	45
No						
4	9237-HQITU	Female	no	No	No	2
Yes						
5	9305-CDSKC	Female	no	No	No	8
Yes						
6	1452-KIOVK	Male	no	No	Yes	22
Yes						
7	6713-OKOMC	Female	no	No	No	10
No						
8	7892-P00KP	Female	no	Yes	No	28
Yes						
9	6388-TABGU	Male	no	No	Yes	62
Yes						
MultipleLines InternetService OnlineSecurity ...						
DeviceProtection \						
0	No phone service		DSL		No	...
No						
1	No		DSL		Yes	...
Yes						
2	No		DSL		Yes	...
No						
3	No phone service		DSL		Yes	...
Yes						
4	No	Fiber optic		No		...
No						
5	Yes	Fiber optic		No		...
Yes						
6	Yes	Fiber optic		No		...
No						
7	No phone service		DSL		Yes	...
No						
8	Yes	Fiber optic		No		...
Yes						
9	No	DSL		Yes		...
No						
TechSupport StreamingTV StreamingMovies Contract						
PaperlessBilling \						
0	No	No	No	Month-to-month		
Yes						

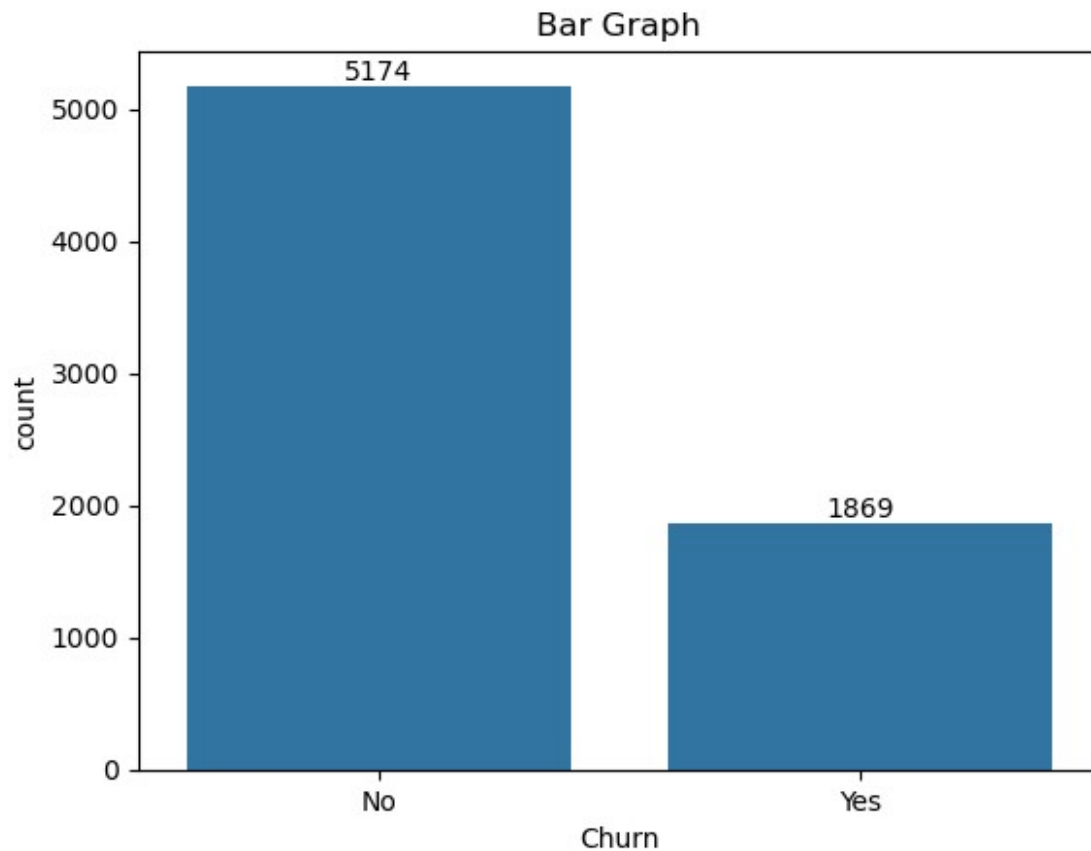
1	No	No	No	One year
No				
2	No	No	No	Month-to-month
Yes				
3	Yes	No	No	One year
No				
4	No	No	No	Month-to-month
Yes				
5	No	Yes	Yes	Month-to-month
Yes				
6	No	Yes	No	Month-to-month
Yes				
7	No	No	No	Month-to-month
No				
8	Yes	Yes	Yes	Month-to-month
Yes				
9	No	No	No	One year
No				

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No
1	Mailed check	56.95	1889.5	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes
5	Electronic check	99.65	820.5	Yes
6	Credit card (automatic)	89.10	1949.4	No
7	Mailed check	29.75	301.9	No
8	Electronic check	104.80	3046.05	Yes
9	Bank transfer (automatic)	56.15	3487.95	No

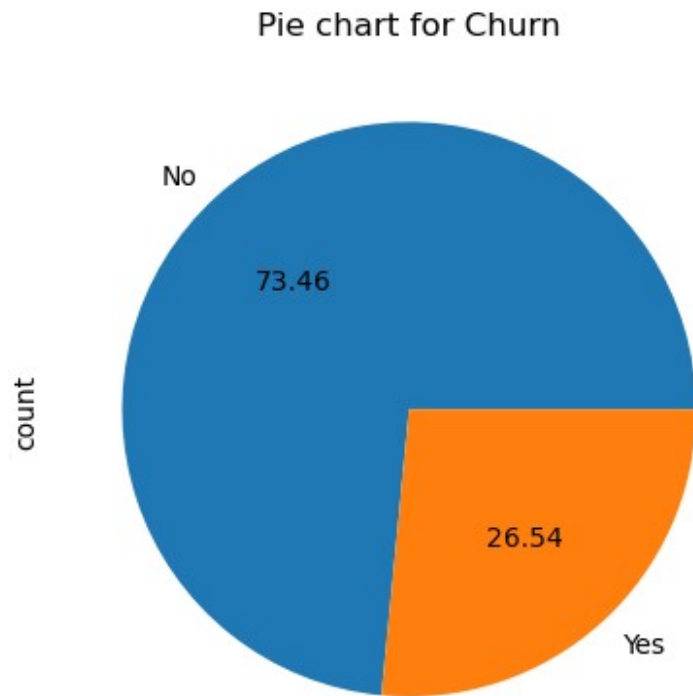
[10 rows x 21 columns]

Understanding how many of customer are churn through bar graph

```
ax=sns.countplot(x='Churn',data=df)
ax.bar_label(ax.containers[0])
plt.title('Bar Graph')
plt.show()
```

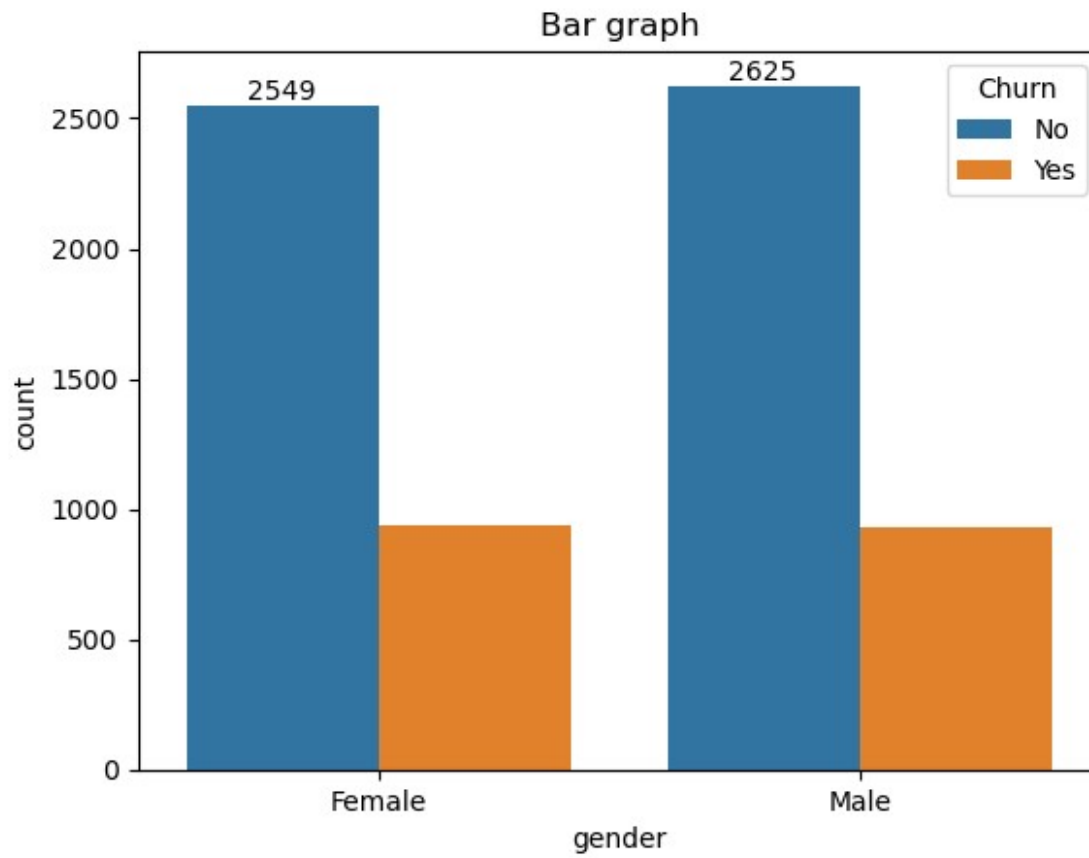


```
#understanding through pie chart  
# Through this we can get that 26% are yes and 74% are No  
df['Churn'].value_counts().plot(kind='pie', autopct='%.2f')  
plt.title('Pie chart for Churn')  
plt.show()
```

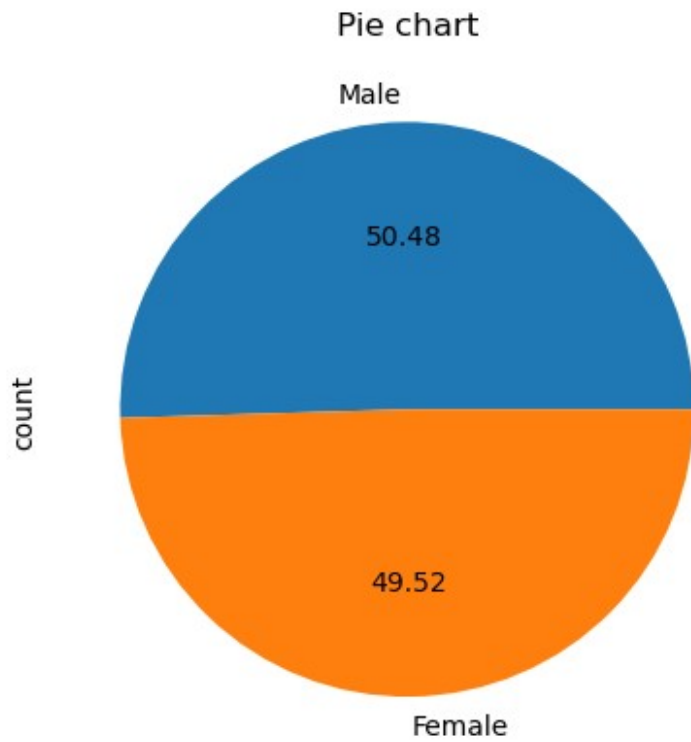


Lets explore for Gender and understanding how many customer are with them

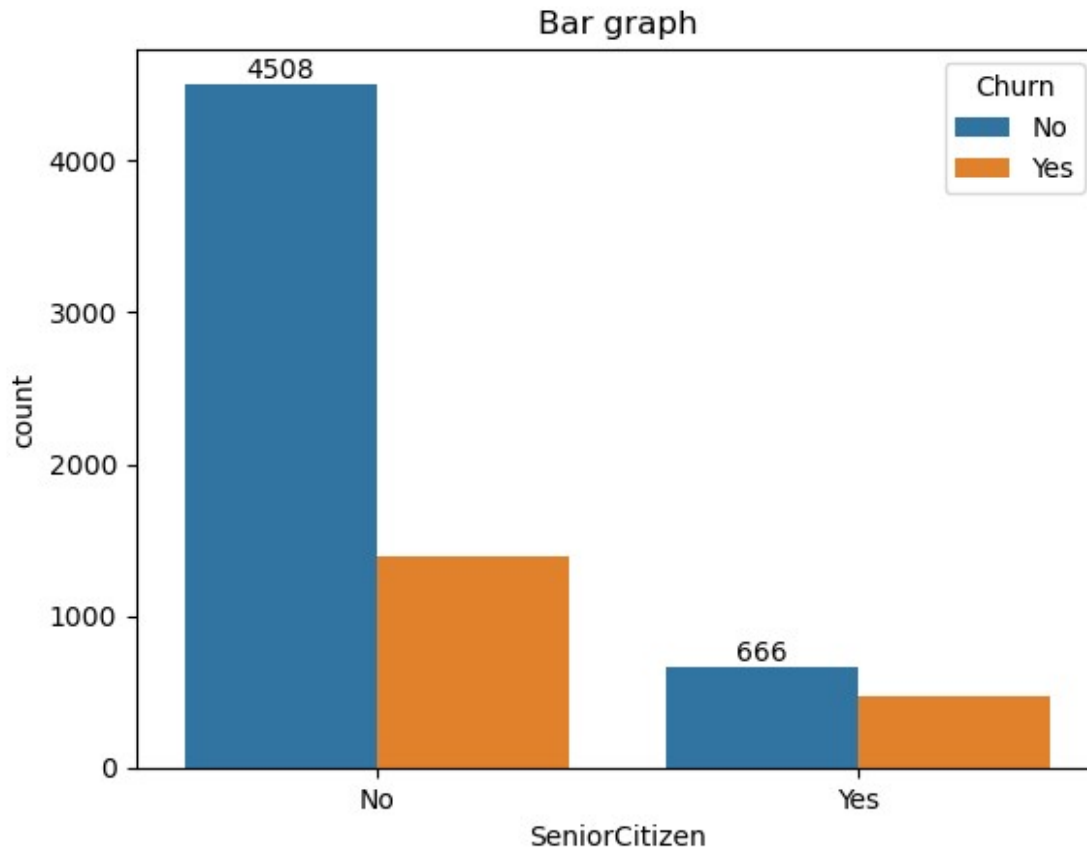
```
ax=sns.countplot(x='gender',data=df,hue='Churn')
ax.bar_label(ax.containers[0])
plt.title('Bar graph')
plt.show()
```



```
#lets visualizing through pie charts
df['gender'].value_counts().plot(kind='pie',autopct='%.2f')
plt.title('Pie chart')
Text(0.5, 1.0, 'Pie chart')
```



```
# Lets explore for senior citizen
ax=sns.countplot(x='SeniorCitizen',data=df,hue='Churn')
ax.bar_label(ax.containers[0])
plt.title('Bar graph')
plt.show()
```



Churn by senior citizen(Stack bar chart)

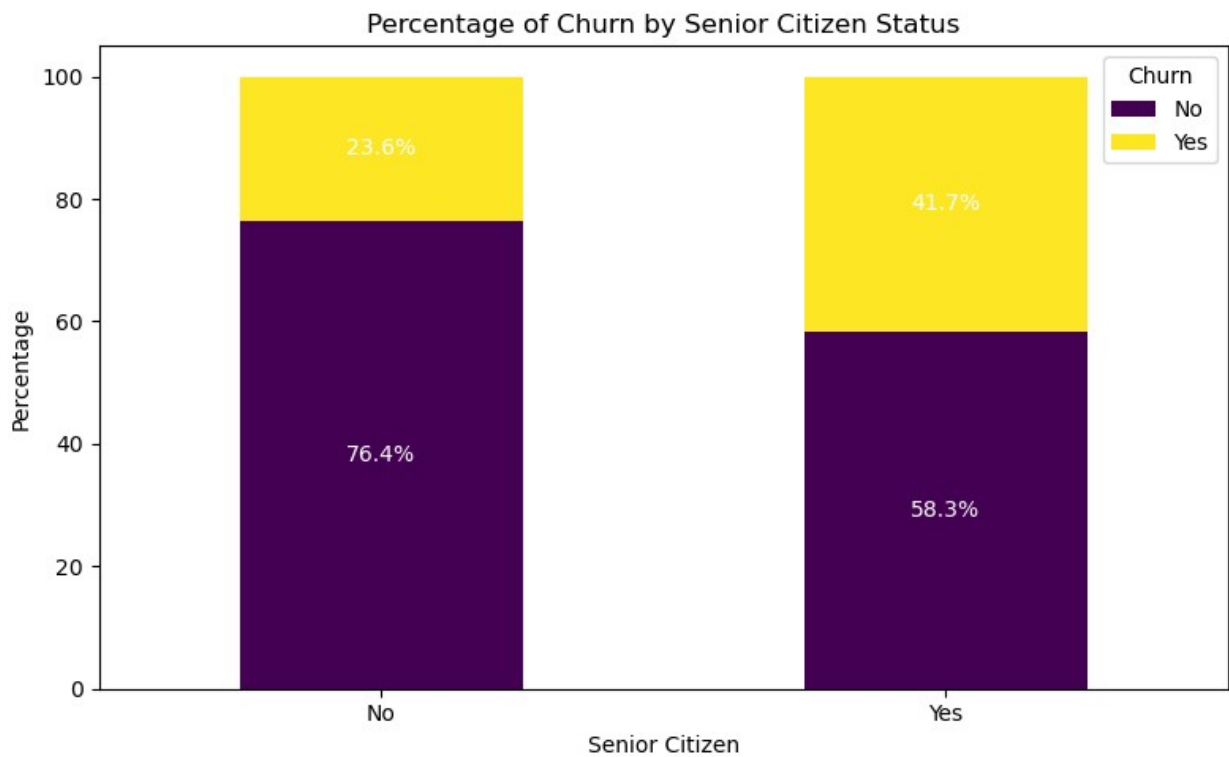
```
# Create a crosstab of counts
ct = pd.crosstab(df['SeniorCitizen'], df['Churn'])

# Convert counts to percentages
ct_percent = ct.div(ct.sum(axis=1), axis=0) * 100

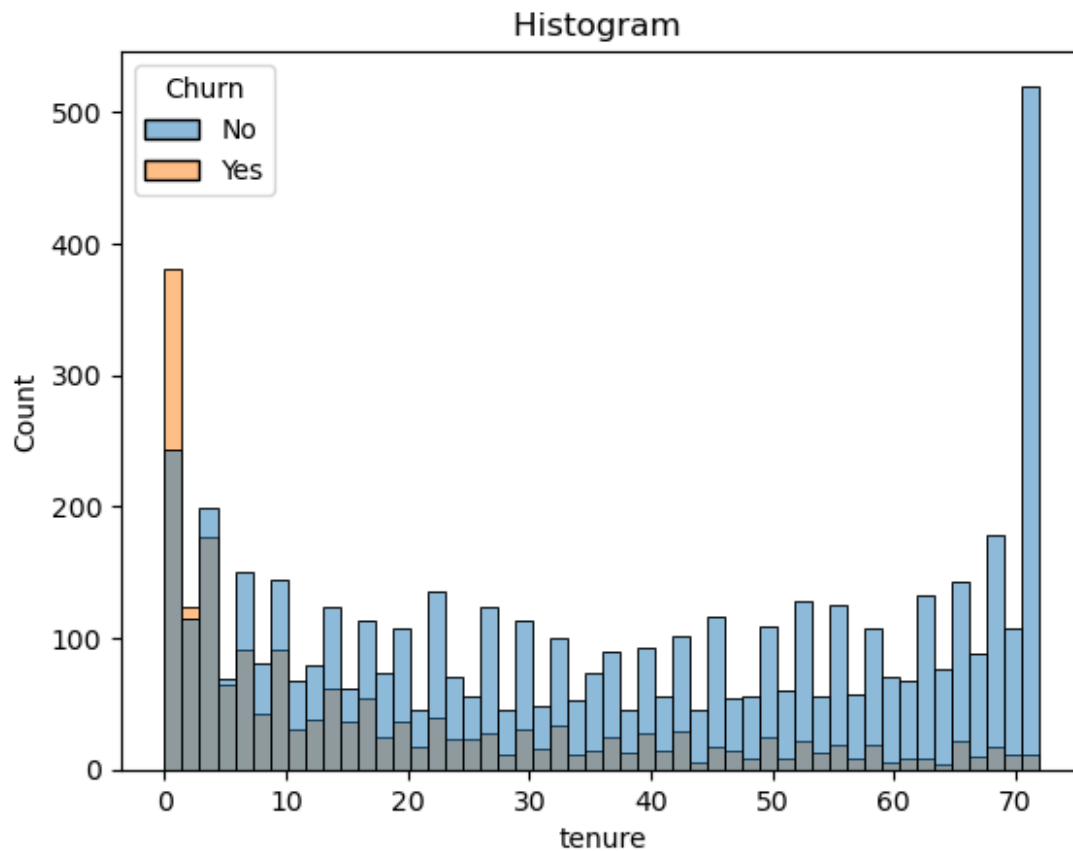
# Plot the stacked bar chart
ax = ct_percent.plot(kind='bar', stacked=True, colormap='viridis',
figsize=(8,5))

# Add percentage labels
for i, row in enumerate(ct_percent.values):
    cum_sum = 0
    for j, val in enumerate(row):
        if val > 0:
            ax.text(i, cum_sum + val / 2, f'{val:.1f}%', ha='center',
va='center', color='white', fontsize=10)
            cum_sum += val
```

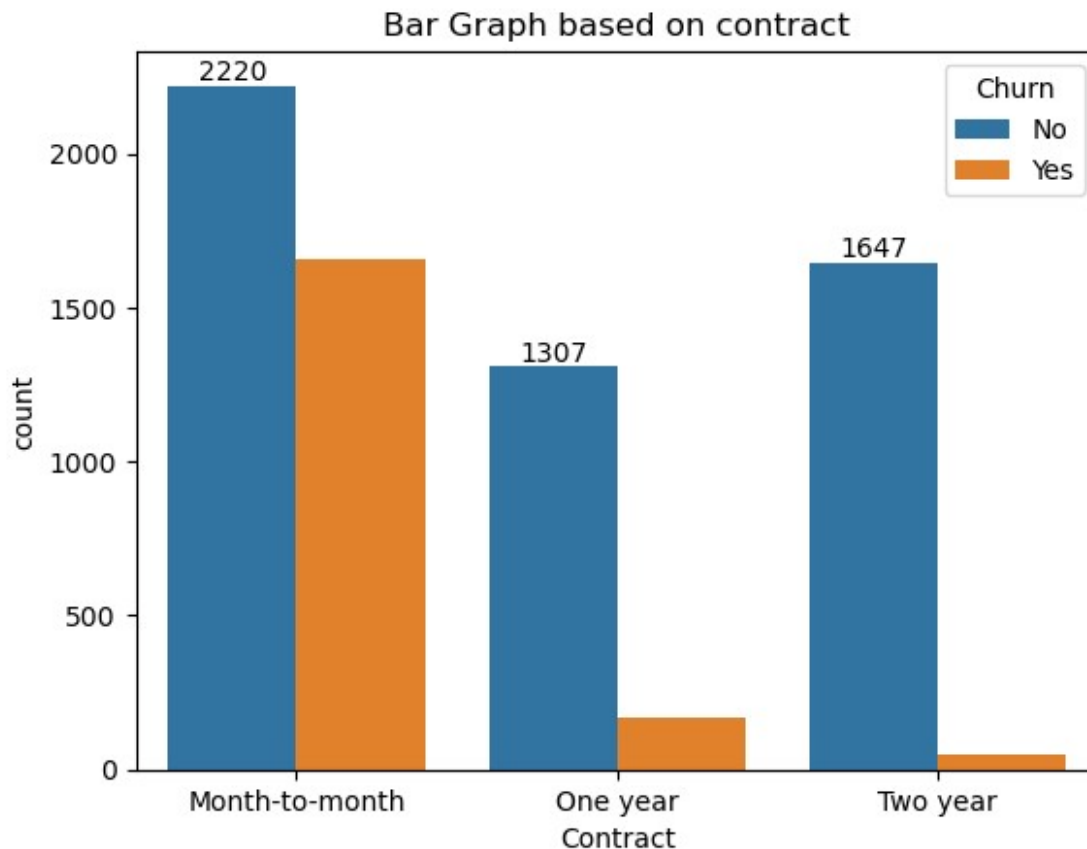
```
plt.title('Percentage of Churn by Senior Citizen Status')
plt.xlabel('Senior Citizen')
plt.ylabel('Percentage')
plt.xticks(ticks=[0,1], labels=['No', 'Yes'], rotation=0)
plt.legend(title='Churn')
plt.tight_layout()
plt.show()
```



```
# Histogram for Tenure and churn
# we observe that those people who uses our services and whose who
leave early ie short term
sns.histplot(data=df,x='tenure',bins=50,hue='Churn')
plt.title('Histogram ')
plt.show()
```



```
#count of people based on contract  
ax=sns.countplot(x='Contract',data=df,hue='Churn')  
ax.bar_label(ax.containers[0])  
plt.title('Bar Graph based on contract')  
plt.show()
```



```
df.columns.values
array(['customerID', 'gender', 'SeniorCitizen', 'Partner',
      'Dependents',
      'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
      'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
      'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract',
      'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges',
      'TotalCharges', 'Churn'], dtype=object)
```

Summary of below subplots

The subplots display count distributions for various telecom services. Most customers have a phone service and fiber optic internet. A large portion does not use OnlineSecurity, TechSupport, or DeviceProtection. Many customers also lack OnlineBackup and do not stream TV or movies. "No internet service" appears as a consistent third category in multiple features, indicating its impact across other service usages.

```
# Your target columns
columns = [
```

```

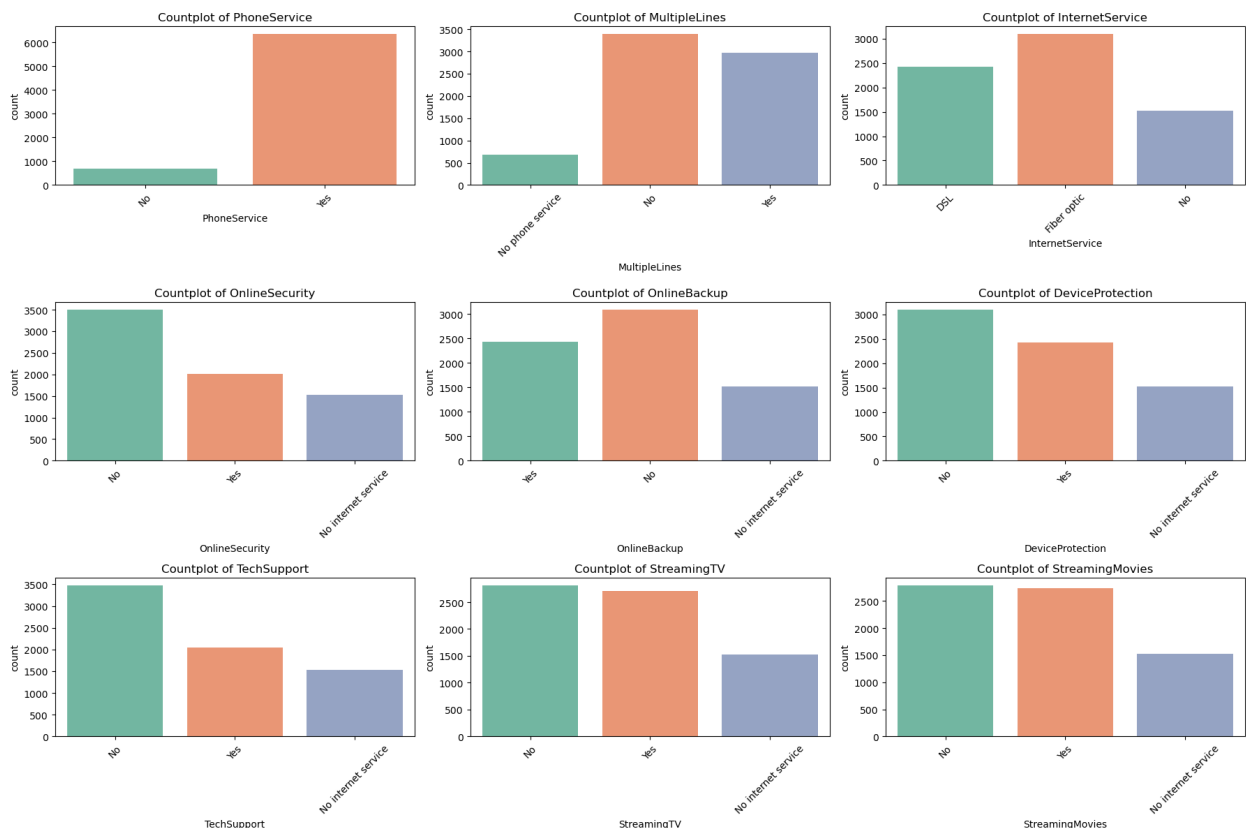
'PhoneService', 'MultipleLines', 'InternetService',
'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
'TechSupport', 'StreamingTV', 'StreamingMovies'
]

# Create subplots (3 rows x 3 columns for 9 plots)
fig, axes = plt.subplots(nrows=3, ncols=3, figsize=(18, 12))
axes = axes.flatten() # Flatten to easily loop

# Loop over each column and create countplots
for i, col in enumerate(columns):
    sns.countplot(data=df, x=col, hue=col, ax=axes[i], palette='Set2',
legend=False)
    axes[i].set_title(f'Countplot of {col}', fontsize=12)
    axes[i].tick_params(axis='x', rotation=45)

# Adjust layout to prevent overlapping
plt.tight_layout()
plt.show()

```



```

#lets look for payment method
plt.figure(figsize=(6,6))
ax=sns.countplot(x='PaymentMethod',data=df,hue='Churn')

```

```
ax.bar_label(ax.containers[0])  
plt.title('Bar Graph for payment churned customer')  
plt.tight_layout()  
plt.xticks(rotation=45)  
plt.show()
```

